Sistema Integrado de Servicios Estudiantiles

Proyecto final · Android nativo · Jetpack Compose · MVVM

Idea central para explicar al profesor

Construimos un portal con tres módulos conectados por un mismo estado: el estudiante guarda su perfil, liquida cursos libres y consulta el total acumulado. El ViewModel decide las reglas; Compose representa sus resultados.

Módulo	Qué demuestra
01 · Perfil académico	Validación de entradas y generación de correo institucional simulado.
02 · Matrícula libre	Selección de curso, beca del 50 %, recargo Icetex del 5 % y bloqueo sin perfil.
03 · Historial financiero	Lista de liquidaciones y total acumulado reactivo.

Qué contiene esta guía

Preparación del proyecto, explicación del código, decisiones de diseño, capturas del emulador, guion de demostración, pruebas ejecutadas y preguntas de sustentación.

Alcance de la entrega

Aplicación académica de un solo perfil, con almacenamiento local. Los precios son de ejemplo y no se conecta con UTS ni con Icetex. No realiza pagos reales.

Preparado para la entrega del 9 de noviembre de 2026 (America/Bogota). Datos de prueba ficticios: Ana Torres, código 2026001.

Paso a paso: preparación

1. Crear un proyecto independiente

El código se creó en la carpeta PARCIAL 3 CORTE APLICACIONES MOVILES, sin modificar el proyecto GimnasioUTS que ya estaba abierto. Se agregó el módulo Android **app**, su manifiesto y el Gradle Wrapper.

Por qué: Mantener un proyecto reproducible y conservar los trabajos anteriores.

2. Configurar herramientas compatibles

Se fijaron Gradle 8.9, AGP 8.7.3, Kotlin 2.0.21, SDK 35 y Compose BOM 2024.12.01. La compilación usa JDK 17; Android Studio también puede usar JDK 21. Se redujo Gradle a dos trabajadores y 1 GB de heap.

Por qué: El JDK 25 de Android Studio provocó incompatibilidad. Limitar los procesos ayudó en un equipo de 8 GB de RAM.

3. Separar responsabilidades

Se crearon los tres paquetes obligatorios. MainActivity construye el repositorio y entrega el ViewModel a la aplicación.

Por qué: Evitar cálculos y acceso al almacenamiento dentro de las pantallas.

```
data/       Models.kt, StudentRepository.kt
            LocalStudentRepository.kt
viewmodel/  PortalViewModel.kt
ui/         ProfileScreen.kt, TuitionScreen.kt
            HistoryScreen.kt, PortalApp.kt
            Components.kt, Theme.kt, Accessibility.kt
```

Recorrido de una acción
Usuario pulsa Guardar perfil → la UI envía un evento → el ViewModel valida → el repositorio guarda → cambia StateFlow → Compose muestra el nuevo estado.

Para ejecutar otra vez: abrir esta carpeta en Android Studio, sincronizar Gradle, elegir la configuración app y el emulador Pixel_10a, y presionar Run.

Paso a paso: perfil



4. Definir el modelo Student
Contiene nombre, código, facultad y correo. El código es texto para conservar ceros iniciales.

5. Capturar y validar
ProfileScreen muestra OutlinedTextField y un menú de facultad. saveProfile() exige nombre de 2 a 80 caracteres, código de 5 a 12 dígitos y facultad seleccionada.

6. Generar el correo
El ViewModel recorta espacios, elimina tildes, usa minúsculas y une nombre, código y dominio. Ejemplo: José Pérez / 0012345 produce jose.perez.0012345@uts.edu.co.

Por qué se hace en el ViewModel
La misma regla se puede probar sin dibujar la pantalla. La UI no decide si el perfil es válido.

Qué mostrar
Guardar primero con campos vacíos. Después completar datos y mostrar el correo y la tarjeta de perfil activo.

Decisión explícita: los límites del nombre y código son reglas del proyecto, no normas oficiales verificadas de UTS. Un borrador editado no cambia el perfil guardado hasta pulsar Guardar perfil.

Código para abrir: viewmodel/PortalViewModel.kt, línea 49; ui/ProfileScreen.kt.

Paso a paso: liquidación



7. Preparar el catálogo
Inglés Técnico: $200.000. Programación Web: $300.000. Ciberseguridad: $400.000. Son valores simulados en COP.

8. Conectar los controles
El menú llama selectCourse(). El Checkbox llama setScholarship(). El Switch llama setIcetex(). Cada evento recalcula la cotización en el ViewModel.

9. Aplicar los porcentajes
Primero se descuenta la beca. Después se calcula el recargo sobre el valor restante a financiar. Esta interpretación queda visible en la pantalla.

Ejemplo con ambos controles
Base: $200.000.
Beca: -$100.000.
Recargo: $5.000.
Total: $105.000.

Por qué Long
Con estas tarifas todos los resultados son pesos enteros. Se evitan imprecisiones de Float/Double. Para tarifas con centavos se necesitaría una regla de redondeo.

```
descuento = si hay beca: base / 2; si no: 0
recargo = si hay Icetex: (base - descuento) * 5 / 100
total = base - descuento + recargo
```

Código para abrir: viewmodel/PortalViewModel.kt, línea 86; ui/TuitionScreen.kt.

Paso a paso: historial



10. Registrar una liquidación
liquidate() verifica el perfil y crea Enrollment con identificador UUID, fecha, curso, condiciones, copia del estudiante e importes.

11. Consolidar el total
El ViewModel suma quote.total de cada registro después de agregar uno nuevo. HistoryScreen solo muestra accumulatedTotal.

12. Dibujar una lista eficiente
LazyColumn presenta los registros y usa el ID como clave estable. Los más recientes aparecen primero.

13. Conservar la información
LocalStudentRepository guarda perfil e historial en SharedPreferences como un documento JSON. No requiere Internet.

Por qué guardar una copia
Editar después el nombre o las condiciones del formulario no modifica lo que ya se había liquidado.

Persistencia y límites: apply() escribe a disco de forma asíncrona. El perfil y las liquidaciones se recuperan al iniciar de nuevo; los borradores no se guardan tras cerrar el proceso. Para un historial grande o requisitos transaccionales, el siguiente paso sería Room.

Código para abrir: viewmodel/PortalViewModel.kt, línea 93; data/LocalStudentRepository.kt; ui/HistoryScreen.kt.

Cómo defender la arquitectura

MVVM: quién hace qué

Capa	Responsabilidad	No debe hacer
Model / data	Definir entidades, catálogo y almacenamiento local.	Dibujar pantallas.
ViewModel	Validar, generar correo, calcular importes y producir estado.	Referenciar vistas o controles Android.
View / ui	Mostrar el estado y enviar eventos. Formatear textos y fechas.	Calcular descuentos o leer SharedPreferences.

StateFlow encapsulado y flujo unidireccional

```
private val _uiState = MutableStateFlow(PortalUiState())
val uiState: StateFlow<PortalUiState> = _uiState.asStateFlow()

// Dentro de cada una de las tres pantallas:
val state by viewModel.uiState.collectAsStateWithLifecycle()
```

El estado mutable solo se cambia dentro del ViewModel. La pantalla observa una versión de solo lectura y llama funciones para solicitar cambios. collectAsStateWithLifecycle() adapta la colección al ciclo de vida de la pantalla, evitando mantenerla activa innecesariamente en segundo plano.

Dos niveles de protección

El botón Liquidar usa canLiquidate para estar deshabilitado sin perfil guardado. Además, liquidate() retorna si student es nulo. Así la regla también se cumple cuando el método se invoca directamente, como ocurre en las pruebas.

Un ViewModel compartido para los tres módulos

Las pantallas pertenecen al mismo portal y necesitan el mismo perfil e historial. Un ViewModel con alcance de Activity evita estados duplicados. rememberSaveable conserva la pestaña; el ViewModel conserva los datos durante una recreación de Activity.

Material Design 3 y accesibilidad

Se usaron Card, Column, Row, LazyColumn, OutlinedTextField, ExposedDropdownMenuBox, Checkbox y Switch. Hay tema claro/oscuro, desplazamiento vertical y etiquetas accesibles en los controles de beca e Icetex.

Guion de demostración práctica

Duración sugerida: 6 a 8 minutos. Usa datos ficticios.

Paso	Acción en el emulador	Qué explicar
1	Abrir Mi UTS en una instalación sin datos. Entrar a Historial.	El total empieza en cero y la lista tiene un estado vacío.
2	Ir a Matrícula y buscar Liquidar.	El botón está bloqueado porque todavía no existe un perfil guardado.
3	Ir a Perfil. Pulsar Guardar perfil sin completar campos.	Los errores provienen del ViewModel.
4	Escribir Ana Torres, 2026001 e Ingenierías. Guardar.	El correo se genera automáticamente y se habilita la liquidación.
5	Elegir Inglés Técnico. Activar beca e Icetex.	Base $200.000, descuento $100.000, recargo $5.000 y total $105.000.
6	Pulsar Liquidar y abrir Historial.	La nueva matrícula aparece y el acumulado se actualiza.
7	Liquidar Programación Web sin beca ni Icetex.	Se suman $300.000; el acumulado es $405.000 si solo hay estas dos matrículas.
8	Cerrar y abrir la aplicación. Mostrar los paquetes en Android Studio.	La persistencia conserva registros; las reglas están separadas de la UI.

Atención durante la sustentación: el emulador de esta entrega ya contiene el registro ficticio creado por la prueba. Para demostrar el estado vacío usa otro emulador o borra únicamente los datos de Mi UTS, sabiendo que se perderán ese perfil e historial locales. No pulses varias veces Liquidar salvo que quieras generar varias matrículas.

Frase de cierre sugerida

“La interfaz reacciona a un estado que controla el ViewModel. Por eso puedo cambiar de pantalla sin perder los datos, probar las reglas sin una interfaz y reutilizar el almacenamiento mediante una interfaz de repositorio”.

Verificación y resultados

Resultado comprobado
APK debug compilado. 11 pruebas unitarias y 2 pruebas instrumentadas aprobadas, sin fallos. Análisis lint: 0 errores y 6 advertencias: cinco de versiones más recientes de dependencias y una sobre reglas de extracción de datos en Android 12+. Ejecución en Pixel_10a con Android 17 / API 37.

Evidencia	Qué verifica
PortalViewModelTest	Perfil obligatorio, campos vacíos/incorrectos, normalización del correo y ceros iniciales.
12 combinaciones financieras	Los tres cursos con ninguna condición, solo beca, solo Icetex y ambas.
Integridad de estado	Acumulado correcto, historial anterior inmutable, recarga desde repositorio y errores de lectura/escritura.
PortalFlowTest	Navegación real, bloqueo, validaciones, registro, controles, historial y recreación de Activity.
Repositorio local real	Serialización y lectura del perfil y liquidaciones en un archivo de prueba aislado.

Cómo reproducir las verificaciones

```
.\gradlew.bat assembleDebug testDebugUnitTest lintDebug
.\gradlew.bat connectedDebugAndroidTest
```

Configurar antes JAVA_HOME con JDK 17 o 21. Las pruebas instrumentadas requieren un emulador encendido y el recorrido completo espera datos iniciales vacíos. scripts/verificar.ps1 automatiza los comandos; usar -DeviceTests para incluir el dispositivo.

Ubicación de reportes

Pruebas unitarias: app/build/reports/tests/testDebugUnitTest/index.html.
Pruebas Android: app/build/reports/androidTests/connected/debug/index.html.
Lint: app/build/reports/lint-results-debug.html.
Capturas: docs/evidencias/.

Límite de la verificación: se probó el emulador disponible, no una matriz de teléfonos físicos. No se evalúan servidores ni transacciones bancarias porque la aplicación es una simulación.

Preguntas para la sustentación

¿Por qué MVVM?

Separa presentación, reglas y datos. Permite cambiar una pantalla sin reescribir el cálculo, y probar el ViewModel con un repositorio en memoria.

¿Por qué el MutableStateFlow es privado?

Para impedir que la UI cambie el estado sin pasar por las reglas. StateFlow público permite observarlo.

¿Por qué no calcular el total dentro de un Composable?

La UI puede recomponerse muchas veces. El cálculo es una regla de negocio centralizada y comprobable en el ViewModel.

¿Por qué no se perdió el perfil al recrear la pantalla?

El ViewModel tiene alcance de Activity y sobrevive a cambios de configuración. Al iniciar un proceso nuevo, el repositorio recupera los datos guardados.

¿Por qué el recargo es $5.000 y no $10.000 en el ejemplo?

Se aplica el 5 % a los $100.000 que quedan después de la beca. El enunciado no fijó el orden; la interpretación está documentada y visible.

¿Qué mejorarías en una versión real?

Autenticación, backend institucional, base de datos Room, protección de datos y validación oficial de tarifas. Este ejercicio conserva un único perfil local y no procesa pagos.

Entrega del código

El repositorio local incluye código, Gradle Wrapper, README, pruebas y evidencias. El APK de demostración se entrega aparte. Repositorio privado en GitHub: https://github.com/msteran/servicios-estudiantiles-uts. El propietario debe dar acceso al profesor para que pueda consultarlo.

Fuentes y referencia técnica

Enunciado entregado por el usuario: “Miniproyecto Final: Sistema Integrado de Servicios Estudiantiles UTS”. Las decisiones adicionales (precios, límites de campos, persistencia y orden de porcentajes) están identificadas en esta guía y en el README.

Android Developers: recomendaciones de arquitectura
Android Developers: estado en Jetpack Compose
Android Developers: corrutinas y ciclo de vida